FR Portal

AWS ISV Fund Request Portal

Complete Project Documentation

Architecture · Workflows · Database · Deployment · Known Issues

Document Guide

This document is written for someone who has never touched this project before — a new developer, a reviewer, or a teammate picking up maintenance. It covers what the system does, why it is built the way it is, how every major piece works internally, and what is genuinely left to do. Sections build on each other: read Sections 1–4 first for the big picture before diving into the deep-dive sections on the Excel sync engine and deployment, which assume that context.

1. Executive Summary

The FR Portal (AWS ISV Fund Request Portal) is a web application built for Trianz's FundingOps team to manage AWS co-sell funding requests end to end — from a seller submitting a request, through FundingOps review and tracking, to the final funding decision as recorded by AWS. It replaces a manual, spreadsheet-driven process with a structured workflow, role-based access, and an automated sync that pulls AWS's own decisions directly into the portal from the team's existing Excel tracker.

The system is fully built and deployed. As of the latest handoff, the core application (submission, review, dashboard, admin, audit log), Azure AD single sign-on, the Excel sync engine, and real email notifications are all live in production. The only genuinely outstanding item is finishing the automated SharePoint connection for the Excel sync (a manual upload path already covers the same need in the meantime).

1.1 What Problem It Solves

- Before this portal, fund requests and their AWS decisions lived in spreadsheets and email threads, with no single source of truth and no audit trail of who changed what.
- Sellers had no self-service way to submit a request or track its status.
- FundingOps had no dashboard view across all requests, and reconciling the portal against AWS's own Excel tracker was a fully manual exercise.
- There was no automatic notification when a request was submitted or its status changed — people found out by asking.

1.2 What It Does Now

- A seller submits a structured fund request through a web form, covering all 45 relevant fields — customer details, Trianz team, eligibility criteria, and scope.
- FundingOps sees every request on a dashboard with Kanban-style stage tracking, KPIs, and filters, and can edit the fields that belong to them.
- The team's existing "AWS Master Billing" Excel tracker — which holds AWS's own decisions on each fund — is synced directly into the portal, either by uploading the file or, once fully configured, automatically on a schedule from SharePoint.
- Real email notifications go out automatically when a request is submitted and when its status changes.
- An admin panel lets a super admin manage user roles, and every change in the system is recorded in an audit log.

2. Technology Stack

Layer	Technology	Notes
Frontend	React 19 + Vite + TypeScript	Single-page application; Tailwind CSS for styling

Layer	Technology	Notes
Backend	NestJS (Node.js) + TypeScript	REST API under /api/v1
Database	PostgreSQL (Azure Flexible Server)	Accessed via Prisma ORM
Authentication	Azure AD (Microsoft Entra ID) + MSAL	Single sign-on with company accounts
Hosting — Backend	Azure App Service (Linux)	Node.js runtime
Hosting — Frontend	Azure Static Web Apps	Purpose-built for SPA hosting
Excel Sync	Azure Function App + shared TypeScript engine	Timer-triggered, plus a manual upload path in the portal
Email	Azure Communication Services (ACS)	Email Communication Service with a connected domain
Source Control	GitLab (self-hosted, git.trianz.com)	Single monorepo for frontend, backend, and the Function app

3. System Architecture

The system has four independently deployed pieces, all living in the same Azure resource group (fr-portal-rg) under the Corp IT subscription:

1. Frontend (Static Web App) — the React application users interact with in the browser.
2. Backend (App Service) — the NestJS API that owns all business logic and the only thing that talks directly to the database.
3. Database (PostgreSQL Flexible Server) — the single source of truth for all application data.
4. Excel Sync Function (Function App) — a separate, scheduled service that reads the AWS Master Billing workbook and writes funding updates directly to the database.

3.1 How a Request Flows Through the System

- A seller logs in via Azure AD SSO in the browser → the frontend calls the backend API with a bearer token.
- The backend verifies the token, checks the user's role, and writes the new fund request to PostgreSQL.
- The backend queues an email send via Azure Communication Services — this happens in the background, not blocking the user's submission.
- FundingOps logs in, sees the request on their dashboard (fetched from the same backend API), and can edit the fields that belong to their role.
- Separately, on its own schedule (or via manual upload), the Excel Sync engine reads the AWS Master Billing workbook, matches each fund to the right request, and updates the AWS-side fields (stage, status, PO number, amounts) directly in the database.
- Every change — a submission, an edit, a sync update — is recorded in the audit log (the sync_log table and the application's own audit trail).

3.2 Why Two Separate Callers for Excel Sync

The Excel sync logic (parsing, aggregating, matching) lives in exactly one place in the codebase and is used by two different callers: the portal's manual upload feature (which writes through the backend's normal Prisma database layer) and the scheduled Azure Function (which writes directly via a PostgreSQL client, independent of the backend being up). Keeping one shared engine means both paths always produce identical results — there is no risk of the

dashboard disagreeing with what an overnight automated sync wrote, because they are not two different implementations, just two different triggers for the same code.

4. User Roles & Permissions

Role	Can Do	Cannot Do
seller	Submit a new fund request; view and read all fields of their own requests	Edit a request after submitting it; see other sellers' requests; access the dashboard, admin panel, or Excel sync page
funding_ops	View all requests on the dashboard; edit the FundingOps-owned fields on any request (status, tech brief, QA checklist, notes); upload Excel files to sync AWS data	Change user roles or deactivate users
super_admin	Everything funding_ops can do, plus manage users (change roles, activate/deactivate) and view the full audit log	—

Role checks are enforced on the backend (the API refuses a request from the wrong role even if the frontend page were somehow reached directly), not only by hiding pages in the UI — this matters because the Excel upload endpoint, for example, writes AWS funding figures across every request in the system, so it must never trust the frontend alone to gate access to it.

A seller can read every field of their own request at any time, but once a request is submitted, they can no longer edit any field on it — this prevents a seller from altering numbers after FundingOps has started reviewing them. Fields that come from the Excel sync (AWS Stage, AWS Status, AWS Fund Stat) are read-only everywhere in the portal for everyone, because they must always reflect exactly what the Excel tracker says — letting anyone hand-edit them would create two conflicting sources of truth.

5. Core Workflows

5.1 Seller: Submitting a Fund Request

5. Seller logs in with their Trianz Microsoft account.
6. Fills out the new request form, covering customer details, deal type, engagement type (Infra / Modernization / Migration — a request can be more than one at once), the Trianz team involved, and eligibility criteria.
7. Depending on which engagement types are selected, additional scope fields appear — VM counts for Infra, custom app counts and pathways for Modernization, VM counts for Migration.
8. On submit, the backend validates all required fields are present and returns a clear, readable list of anything missing rather than a generic error.
9. Once saved, two emails go out: one to the seller confirming their submission, and one to the FundingOps distribution list alerting them a new request needs review.
10. The request immediately appears on the FundingOps dashboard and on the seller's own "My Requests" list.

5.2 FundingOps: Reviewing and Tracking Requests

- The Dashboard shows a Kanban-style view grouped by AWS stage, KPI summary cards, and filters.

- Opening a request's detail page shows all 45 fields; FundingOps can edit the fields that belong to their stage of the process (status fields, tech brief uploaded, QA checklist, custom notes) while the seller-submitted fields and the AWS-sync fields remain visible but read-only.
- Saving an edit triggers a status-change email to the seller if the stage or status actually changed.

5.3 Admin: User Management and Audit Log

- A super admin can view every user, change their role, or deactivate them.
- The system refuses to let an admin deactivate or demote themselves in a way that would leave nobody able to manage the system — an earlier defect that allowed exactly this (permanent admin lockout) was found and fixed.
- The audit log lists every change in the system with who made it and when, sourced from the sync_log table — entries are tagged either api:<email> for a human edit through the portal, upload:<email> for a manual Excel upload, or excel-sync for the automated scheduled sync, so it is always clear which of the three caused a given change.

5.4 Excel Sync (Manual Upload)

11. FundingOps or a super admin opens the "Excel Sync" page from the sidebar and uploads the AWS Master Billing workbook (.xlsx).
12. The system refuses a bad file immediately and explains exactly why — a .csv, a corrupted file, or a workbook missing the expected "Append1" sheet are each called out specifically, rather than a generic failure.
13. The file is parsed, aggregated, and matched against existing requests (see Section 9 for exactly how).
14. The page reports precisely what happened: how many funds were created, updated, already up to date, how many Fund IDs were newly linked, which funds need manual linking (ambiguous matches), and any errors — along with the column mapping it resolved, so a reordered spreadsheet is immediately visible.

5.5 Email Notifications

Trigger	Recipient	Purpose
new_submission_alert	FundingOps distribution list	Alerts the team a new request needs review
submission_confirmation	The submitting seller	Confirms their request was received
status_change	The submitting seller	Notifies them when stage or status changes

Every email includes a direct link back to the specific request in the portal. Sends are queued in the background rather than made part of the user's request — so a slow or failed email can never block someone from submitting a form or saving an edit. A failed send is recorded as such in the notifications table rather than silently disappearing, and is never recorded as a false "sent" — see Section 10 for the real production bug this distinction caught.

6. Authentication & Authorization

The portal uses real Azure AD (Microsoft Entra ID) single sign-on — a user logs in with their actual Trianz Microsoft account, not a separate portal-specific password.

6.1 The Login Flow

15. User clicks "Continue with Microsoft" on the login page.
16. The frontend (using Microsoft's MSAL library) redirects the browser to the real Microsoft login page for the Trianz tenant.

17. After the user enters their credentials, Microsoft redirects back to the portal carrying a signed ID token.
18. The frontend sends that token to the backend's /auth/sso-login endpoint.
19. The backend independently verifies the token is genuine — checking its signature against Microsoft's public keys, and confirming it was issued specifically for this app and this tenant (not forged, not meant for a different application).
20. If this is the user's first login, a user record is created automatically with the default "seller" role; otherwise their existing role is loaded.
21. The backend issues its own short-lived application token (a JWT), which the frontend then uses for every subsequent API call.
22. On every single request afterward, the backend re-checks the user is still active in the database — not just trusting whatever role was baked into the token — so deactivating a user takes effect immediately rather than only after their token happens to expire.

6.2 A Note on App Registration Platform Type

The Azure AD App Registration backing this login must be configured as a "Single-page application," not a "Web" application. This is a real, non-obvious distinction: a browser-based app like this one needs to exchange its login result for a token directly from the browser via JavaScript, which requires a security feature (CORS support on Microsoft's token endpoint) that is only enabled for apps registered as "Single-page application." Registering it as "Web" instead causes login to fail with a cross-origin error (AADSTS9002326) that gives no indication this is the cause — this exact misconfiguration was hit and fixed during deployment.

7. Database Schema

The database has four tables. Below is each one in plain terms, followed by the exact field list for the central fund_requests table.

7.1 Tables Overview

Table	Purpose
users	Every person who can log in — their email, name, role, and whether they are active
fund_requests	The central table — every fund request, all 45+ fields, from both the seller and AWS sync
sync_log	An audit trail of every field change on every request, recording the old value, new value, and who/what made the change
notifications	A record of every email the system has sent or attempted, including its delivery status

7.2 fund_requests — Full Field Reference

Seller-Submitted — Core Details

- requestTitle, dealType, partnerName, customerName, country
- sfdcId (Salesforce opportunity ID) and fundId (assigned by AWS after submission — starts blank)
- engagementTypes — a multi-select array (a single request can be Infra + Modernization + Migration together)

Populated by the Excel Sync — AWS's Own Decision Data

- awsActivity, awsStage, awsStatus, awsFundStat, awsPoNumber, awsExpiry
- fundingSubmissionAmountUsd, finalCashClaimAmountUsd — by design, these two are set to the same aggregated value from the sync

These fields are null until the first Excel sync runs, and are read-only in the portal UI everywhere — AWS's workbook is the single source of truth for them.

Seller-Submitted — Customer & Technical Details

- awsAccountId, awsCalcUrl, sourceWorkloadType, sourceWorkloadLocation, targetDestinationType, competitorConsideration

Seller-Submitted — Trianz Team

- salesPartner, solutionArchitect, fundingOwner, financeOwner, otherStakeholders

Seller-Submitted — Eligibility Criteria

- trianzSi, conciertoSoftwarePartner, opportunityStage, stageGateMet, opportunityValueAbove125k, arrAlignmentConfirmed, roiTargetMet

Scope Fields (conditional on Engagement Type)

- Infra: totalVms, itdInfraUploaded
- Modernization: customApps, itdModUploaded, appPathways, dbPathways
- Migration: migrateVms, itdMigrateUploaded

FundingOps-Owned — Status & Lifecycle

- techBriefUploaded, qaChecklistComplete, conciertoFundingStatus
- submissionDate, awsDecisionDate, awsFrId, rejectionReason, customNotes

7.3 Key Design Decisions in the Schema

- A unique constraint on (sfdcId, fundId) together is the key the Excel sync uses to find the right request — this is deliberate, since the same Salesforce opportunity can have multiple separate AWS funds.
- engagementTypes is stored as a native PostgreSQL array rather than a comma-joined string, specifically so the dashboard can filter on individual engagement types directly in the database.
- An earlier version of the form had a separate, manually-set "AWS Funding Status" field. It was removed deliberately — it duplicated what the Excel sync already writes into awsStage/awsStatus under a different label, and letting anyone hand-type a value there risked contradicting whatever the sync later set. awsStage/awsStatus are now the single source of truth.

8. Excel Sync Engine — How It Actually Works

This is the single largest and most carefully built piece of the system, because it is dealing with real money and a workbook that was not designed with automated parsing in mind. The engine lives in exactly one place — backend/src/excel-sync/ — and is shared by both the manual upload feature and the scheduled Azure Function, so the two can never drift apart.

8.1 The Source Data

The source is the "AWS Master Billing" tracker, specifically its Append1 sheet — a workbook maintained outside the portal by the team, where each row is one billing line for one fund in one month. A single fund frequently spans several rows, because AWS bills per month, per phase, and per workload type.

8.2 Columns Are Matched by Header Name, Never by Position

This was a hard-learned lesson, not a design preference from the start. A newer copy of the real workbook arrived with every column completely reordered — the Amount column, for example, moved from position 16 to position 7. A parser that reads columns by fixed position would have silently written VM counts into funding amounts, with no error at all, because every value is still a number — it would simply be the wrong number. The parser now resolves every column by its header text instead, matching case- and whitespace-insensitively but otherwise exactly (the sheet genuinely contains both "AWS Stage" and "AWS Stage1" as separate columns, so fuzzy matching would silently guess wrong). If a required column cannot be found by name, the sync fails loudly and names exactly which column is missing, what it expected, and every header it actually found — rather than silently producing wrong numbers.

8.3 One Fund, Many Rows — Aggregation Rules

Because a fund can appear across several billing-line rows, an early version of the parser kept only the first row per fund and discarded the rest — silently losing real money (one fund's total was understated by over half a million dollars this way). The rules FundingOps settled on, now implemented exactly, are:

23. A fund's total amount is the sum of all its distinct billing lines.
24. Lines that are identical except for the month count only once (this case belongs to "Manage" billing, which this sync does not track — in practice no funded row ever hits this case, since Manage rows carry no Fund ID at all).
25. There is exactly one portal request per Fund ID.
26. Final Cash Claim is defined as equal to the aggregated billed amount — the two key dashboard numbers are equal by design, not by coincidence.
27. Negative amounts are intentional (they represent credits) and are passed through untouched, not treated as errors.
28. Dropped or cancelled funds still count toward the totals.

8.4 Distinguishing Genuinely Separate Billing Lines

Two further real-data problems had to be solved to get the totals exactly right:

- The sheet sometimes lists what look like duplicate lines that are actually genuinely separate billing tranches, distinguished only by a suffix on the sheet's own internal QC identifier (e.g. "-2", "-3"). Treating these as true duplicates and collapsing them lost real money from the totals.
- The "M" column is not always a simple calendar month — one real fund billed M1 and M2 as two separate migration tranches with completely different amounts and VM counts, not two months of the same thing. Blanking the month field to detect duplicates merged these two tranches together and silently lost one of them entirely.

The fix for both: a billing line's identity is its QC value with the month blanked out, plus its amount — not the month field alone. This was verified by checking that the sum of all fund totals exactly equals the sum of all individual line amounts, to the cent, against the real production workbook.

8.5 Matching a Fund to the Right Portal Request

A fund from the spreadsheet is attached to an existing portal request in two passes, never by guessing:

29. If a request already has a Fund ID recorded, that match wins outright — no further logic needed.
30. Otherwise, if the spreadsheet shows exactly one fund for a given Salesforce opportunity (SFDC ID) and exactly one existing request is waiting without a Fund ID for that same opportunity, the two are matched and the Fund ID is written back onto that request — this matters because AWS only issues a Fund ID after a request is submitted, so without this step that request would stay permanently unlinked.

31. Everything else — an SFDC ID with several funds and no way to tell them apart automatically — is listed in the sync report for a human to resolve manually. The system never guesses in an ambiguous case, because a wrong guess would attach AWS funding data to the wrong request with nothing downstream ever revealing the mistake.

8.6 Funds With No Existing Request

Many funds in the real workbook predate the portal entirely — no seller ever raised them, because they already existed in AWS before this system was built. For these, the sync automatically creates a new request rather than leaving the dashboard empty, owned by an inactive, non-human placeholder account ("Unassigned (Excel import)") that can never sign in. Country and engagement type are deliberately left blank on these imported requests, for a person to fill in later — this was an explicit decision agreed with FundingOps, not an oversight.

8.7 What a Re-Sync Does

Situation	Result
Fund already in the portal, values unchanged	Nothing is written — a re-sync of identical data changes nothing
Fund already in the portal, a value changed	Updated in place; the row count does not grow
Fund with no matching request	A new request is created (see 8.6)
Fund removed from the spreadsheet entirely	The portal request is NOT deleted — this is deliberate, so a row accidentally deleted from a spreadsheet can never wipe out real funding history

8.8 Verified Production Numbers

On the real "AWS Master Billing" workbook: 125 individual billing lines resolved into 119 distinct funds, totaling $8,803,184.40 — matching the source workbook exactly to the cent. All 119 were imported as portal requests on the first sync; re-uploading the identical file afterward correctly produced zero creates and zero updates.

9. Email Notifications — How It Actually Works

Email is sent through Azure Communication Services (ACS). The integration was deliberately built so that a misconfiguration can never take the application down — if ACS is not properly configured, the system falls back to logging what it would have sent rather than crashing or blocking real users.

9.1 Mock Mode vs. Live Mode

A single environment variable, ACS_MOCK, controls this. Unless it is set to exactly the string "false", the system stays in mock mode — a deliberate choice, so that a typo or a missing value can never accidentally and silently start sending real email. In mock mode, every would-be send is logged with its recipient and subject, and recorded in the database with status "mocked" (never "sent") — so the audit trail can never lie about whether an email genuinely went out.

9.2 A Real Production Bug Worth Understanding

When real sending was first switched on, submissions succeeded but no email ever arrived — and every system signal said it had worked: the notifications table showed rows marked "sent" with real-looking Azure message IDs, and nothing in the logs mentioned an error. The investigation initially focused on the recipient's side — spam filtering, mail quarantine, sender domain trust — because every signal pointed there.

The actual root cause was in the backend's own code: it asked Azure to send the email and waited for Azure's response, but never checked whether that response said the send had actually succeeded — Azure can return a message ID even for a send that failed, and the code was treating "Azure gave me an ID" as proof of success. The fix: the code now explicitly checks that Azure's status is "Succeeded," and treats anything else as a failure, recording it as "failed" with Azure's own error details attached. After this fix (and a backend restart), email delivered correctly and consistently. The lesson recorded alongside the fix: every signal the application produces is only as trustworthy as the code that produced it — when investigating "why didn't this work," check what the code actually verifies, not just what it logged.

9.3 Recipients

The FundingOps alert recipient list is a comma-separated environment variable (FUNDING_OPS_ALERT_EMAILS) — the service splits it and sends one independent message per address, so a delivery failure to one recipient can never block delivery to the others. Multiple recipients, including addresses on an entirely different mail system, have been added this way with no code changes required.

10. Deployment & Azure Infrastructure

10.1 Azure Resources

Resource	Type	Name
Resource Group	—	fr-portal-rg (Corp IT subscription)
Backend	App Service (Linux, Node.js)	fr-portal-backend
Frontend	Static Web App	fr-portal-frontend
Database	PostgreSQL Flexible Server	fr-portal-db-server, database fr_portal
Excel Sync	Function App	fr-portal-excel-sync-1
Email	Communication Services + Email Communication Service	fr-portal-acs + fr-portal-email

10.2 Why Deployment Is Manual, Not Automatic

Azure's built-in Deployment Center, connected directly to the git repository, was tried first and repeatedly failed — because the project keeps the frontend and backend code in a single repository, Azure's automatic build system kept detecting and building the frontend's build script when deploying the backend, no matter how the folder-targeting setting was adjusted. The reliable alternative that is actually in use: build the application properly in a cloud shell environment, package the already-built output into a zip file, and deploy that package directly. This is slightly more manual, but removes any ambiguity about what gets built and shipped — a redeploy script captures every step so this does not need to be reconstructed from memory each time.

10.3 The CI/CD Pipeline Exists But Is Not Usable As-Is

A GitLab CI/CD pipeline configuration exists in the repository, but it is not safe to simply enable: every job is scoped to run only on a branch called "master," while the project's real branch is "main" — meaning the pipeline has never actually run a single job. Separately, the frontend build job has none of the required build-time configuration variables set, so a pipeline-built frontend bundle would silently try to call a local development address instead of the real backend. Both issues would need fixing before this pipeline could be safely turned on.

11. Environment Variables Reference

11.1 Backend (fr-portal-backend)

Variable	Purpose
DATABASE_URL	PostgreSQL connection string
NODE_ENV	Set to "production"
JWT_SECRET	Signs the application's own session tokens
JWT_EXPIRES_IN	Session token lifetime (e.g. 8h)
AZURE_TENANT_ID / AZURE_CLIENT_ID	Identify the Azure AD App Registration used to verify SSO logins
CORS_ALLOWED_ORIGINS	The frontend's URL — must match exactly or the browser blocks requests
ACS_MOCK / ACS_CONNECTION_STRING / ACS_SENDER_ADDRESS	Controls real vs. mock email sending
FUNDING_OPS_ALERT_EMAILS	Comma-separated recipient list for new-submission alerts

11.2 Frontend (build-time only — baked into the bundle at build, not runtime)

Variable	Purpose
VITE_API_BASE_URL	The backend's URL — note the exact name; a similarly-named but wrong variable name once caused the whole frontend to silently call a local address instead of the real backend
VITE_AZURE_CLIENT_ID / VITE_AZURE_TENANT_ID	Same Azure AD App Registration details as the backend, for the login redirect
VITE_REDIRECT_URI	Must exactly match a redirect URI registered in Azure AD, with no extra path

12. Codebase Structure

12.1 Repository Layout

A single monorepo holds three independently deployed applications:

- / (repo root) and /src — the React frontend
- /backend — the NestJS API
- /excel-sync-function — the scheduled Azure Function

12.2 Backend — Key Folders

Folder	Contents
src/auth	Login logic — both the real Azure AD SSO verification and a development-only mock login, which the backend itself refuses to allow when running in production
src/fund-requests	The core API for creating, listing, and editing fund requests, including the role-aware list of which fields each role is allowed to change
src/fund-sync	The manual Excel-upload endpoint used from the portal's Excel Sync page
src/excel-sync	The shared parsing/aggregation/matching engine described in Section 8 — used by both fund-sync and the separate Function app
src/notifications	The ACS email service described in Section 9
src/users	User management for the admin panel
src/audit-log	Read-only API for the audit trail
src/prisma	The database client wrapper
src/common/guards	Role-based access control, enforced on the server for every protected endpoint

12.3 Frontend — Key Folders

Folder	Contents
src/pages/auth	The login page
src/pages/seller	New request form and "My Requests" list
src/pages/dashboard	The FundingOps Kanban dashboard
src/pages/requests	The shared request detail page (used by every role, with different editable fields)
src/pages/admin	User management
src/pages/sync	The Excel upload page
src/context/AuthContext.tsx	Holds the logged-in user and the Microsoft login/logout logic
src/lib/msal.ts	Azure AD MSAL library configuration
src/lib/api.ts	The shared helper every API call goes through
src/components/layout/RequireRole.tsx	Route guard — redirects away from a page a user's role should not see

12.4 Excel Sync Function — Key Folders

Folder/File	Contents
src/functions	The two registered triggers — one HTTP (also accepts a file posted directly) and one timer, running on a schedule
src/db.ts	Direct PostgreSQL access (not through the backend's Prisma client, since this Function runs independently of the backend being up)

Folder/File	Contents
scripts/sync-shared.js	Copies the shared engine from backend/src/excel-sync at build time, so this Function always ships the same logic as the portal upload, never an older copy

13. Known Issues, Gotchas & Lessons Learned

These are real defects and non-obvious platform behaviors that were hit during this project's build and deployment. Recording them here is meant to save whoever works on this next from rediscovering the same thing the hard way.

13.1 Build & Deployment Gotchas

- A missing rootDir/include/exclude setting in the backend's TypeScript build configuration once caused compiled output to land one folder deeper than expected — looked harmless locally, but would have made the actual Azure deployment's startup command fail to find the application entirely.
- Azure Cloud Shell presets NODE_ENV=development. The frontend build tool honors this and will silently produce a larger, development-mode bundle (with extra developer-only features shipped to real users) unless this is explicitly unset before building for production.
- The frontend environment variable must be named exactly VITE_API_BASE_URL. An earlier deploy used a different, similarly-reasonable name, and the frontend silently fell back to a hardcoded local development address — every API call failed in the browser with no obvious cause, since the build itself succeeded without any error.
- A corrupted dependency (a database driver binary) was traced back to the corporate network's security proxy blocking its download — resolved by regenerating it from an Azure Cloud Shell session instead, which is not behind that proxy.
- A TypeScript compile error can silently cause a deployment script to redeploy an old, stale build instead of failing outright, if the script does not check the build step's own exit status — several "fixes" during debugging appeared to do nothing because of exactly this, until the stale build was identified directly.
- An Azure Function App with no properly configured entry-point glob pattern will deploy successfully and report no error, while silently registering zero actual functions — always confirm the expected functions are registered after any Function App deployment, not just that the deploy command exited cleanly.

13.2 Application Defects Found and Fixed

- A "read-only" screen that was still editable in one spot.
- A logout that left one user's unsaved draft visible to the next person who logged in on the same device.
- An administrator able to deactivate or demote themselves in a way that would permanently lock everyone out of user management — now explicitly refused by the system.
- Several Excel-sync defects detailed in Section 8 — positional column reading, losing billing lines, and merging genuinely separate billing tranches.
- The email delivery-status bug detailed in Section 9.2.

13.3 General Lessons Recorded From This Project

The single most repeated lesson across this project: every serious defect found was caught by running the real application against real data, not by reading the code and reasoning about it. Several of these defects passed code review and even passed hand-written tests, because the tests themselves shared the same wrong assumption as the code they were checking. Testing against the actual production workbook — and checking that totals reconcile to the cent — is what caught problems that type-checking and unit tests alone did not.

14. What Is Genuinely Left

Item	State
SharePoint connection for automated Excel sync	The Function App exists with both triggers already coded. The Azure AD App Registration already has the right Graph API permission with admin consent granted — no new IT request needed for that. Still required: the client secret for that app registration, and the specific SharePoint site and file IDs. The manual upload path already covers this need in the meantime, and remains useful afterward as the only way to force an on-demand sync.
CI/CD pipeline	Exists but is not usable as-is — see Section 10.3. Deploys remain manual and scripted until this is fixed.

15. Testing

The project has 306 automated checks, all passing, split across suites that test different layers:

Suite	What It Proves
verify-e2e	The whole API over real HTTP requests, including role checks and validation
aggregate-and-match	Every one of the Excel-sync aggregation and matching rules from Section 8
change-detection	A re-sync of genuinely unchanged data writes nothing to the database
column-mapping	A reordered spreadsheet is still read correctly, and a near-miss header name is correctly refused rather than guessed
upload-sync / upload-e2e	The manual upload feature, against the real production workbook and over real HTTP
lifecycle	The full real-world sequence: upload, refresh, re-upload, a genuine change, and a removed row
dashboard-preview	No field ever displays in the wrong place; blank values render as a dash, never blank or "undefined"
dry-run	The entire sync path run against the real file, confirming money is conserved exactly
sharepoint-path	The scheduled Function's logic matches the manual upload's logic exactly

The suites that require the real production workbook are deliberately kept separate from the rest, because that workbook contains real commercial data and is not committed to the repository.

16. Summary

This project went from "cannot be logged into" to a fully working, production system handling real AWS funding data and sending real email, across a series of focused working sessions. The two consistent themes throughout: real defects were almost always found by testing against the real application and real data rather than by reasoning about the code alone, and every non-obvious platform behavior that caused real time loss has been written down in Section 13 specifically so the next person does not have to rediscover it.